

CHAPTER 02. 사이킷런으로 시작하는 머신러닝

01. 사이킷런 소개와 특징

=) 사이킷런(Scikit-learn)?

- 파이썬 머신러닝 라이브러리 중 가장 많이 사용됨
- 가장 쉽고 효율적인 개발 라이브러리를 제공

=) 사이킷런 특징

- 파이썬 기반의 다른 머신러닝 패키지도 사이킷런 스타일의 API를 지향할 정도로 쉬움
- 머신러닝을 위한 다양한 알고리즘 + 개발을 위한 편리한 프레임워크 & API 제공
- 오래, 매우 많은 환경에서 사용되는 성숙한 라이브러리

—

02. 첫 번째 머신러닝 - 붓꽃 품종 예측하기

=) 붓꽃 데이터 세트

- 4개의 피처를 이용해 붓꽃 품종 분류
 - sepal length
 - sepal width
 - petal length
 - petal width
- 3개의 품종 레이블
 - 0 → Setosa
 - 1 → Versicolor
 - 2 → Virginica
- 붓꽃 데이터 세트 생성
 - load_iris() 이용
 - ML 알고리즘 = 의사 결정 트리(Decision Tree) 알고리즘
 - 이를 구현한 Decision Tree Classifier를 적용

=) 지도학습 (ex. 분류)

과정: 학습 데이터 → 모델 학습 → 테스트 데이터 예측 → 실제 값과 비교 → 성능 평가

=) 사이킷런 패키지 모듈명

- 사이킷런 패키지 내의 모듈명은 **sklearn**으로 시작하는 명명규칙이 있음
 - **sklearn.datasets**: 사이킷런이 자체적으로 제공하는 데이터 세트를 생성하는 모듈 모임
 - **sklearn.tree**: 트리 기반 ML 알고리즘을 구현한 클래스의 모임
 - **sklearn.model_selection**: 학습/검증/예측 데이터로 분리하거나 최적 하이퍼파라미터로 평가하기 위한 다양한 모듈의 모임
 - 하이퍼 파라미터: ML 알고리즘별 최적의 학습을 위해 직접 입력하는 파라미터를 통칭
 - ↳ 이를 통해 ML 알고리즘 성능 튜닝도 가능

—

03. 사이킷런 기반 프레임워크

=) Estimator 이해 및 **fit()**, **predict()** 메서드

- **Classifier**: 분류 알고리즘을 구현한 클래스
- **Regressor**: 회귀 알고리즘을 구현한 클래스
- Classifier + Regressor = Estimator

=) **fit()**

- 학습 데이터를 이용해 모델 학습

=) **predict()**

- 학습된 모델을 이용해 새로운 데이터 예측

=) 내장된 예제 데이터 세트

- 사이킷런에는 예제로 활용 가능한 데이터 세트가 내장되어 있음
- **datasets** 모듈에 있는 여러 **API**를 호출해 만들 수 있음
- 예제용 데이터 세트
 - **datasets.load_boston()**, **datasets.load_breast_cancer()**, **datasets.load_diabetes()**, **datasets.load_digits()**, **datasets.load_iris()**
 - **fetch_covtype()**: 회귀 분석용 토지 조사 자료 / **fetch_20newsgroups()**: 뉴스 그룹 텍스트 자료 / **fetch_olivetti_faces()**: 얼굴 이미지 자료 / **fetch_lfw_people()**: 얼굴 이미지 자료 / **fetch_lfw_pairs()**: 얼굴 이미지 자료 / **fetch_rcv1()**: 로이터뉴스 말뭉치 / **fetch_mldata()**: ML 웹사이트에서 다운로드
- 표본 데이터 생성기
 - **datasets.make_classifications()**, **datasets.make_blobs()**
- 예제용 데이터 세트 키
 - **data**: 피쳐의 데이터 세트
 - **target**: 분류 시 레이블 값 회귀일 때는 숫자 결괏값 데이터 세트
 - **target_name**: 개별 레이블 이름

- `feature_names`: 피쳐 이름
- `DESCR`: 데이터 세트에 대한 설명 + 각 피쳐의 설명

04. Model Selection 모듈 소개

=> `model_selection` 모듈

- 학습 데이터와 테스트 데이터 세트를 분리
- 교차 검증 분할 및 평가
- `Estimator`의 하이퍼 파라미터 튜닝을 위한 다양한 함수/클래스를 제공

=> 학습/데이터 세트 분리

- 테스트 데이터 세트를 이용하지 않고 학습 데이터 세트로만 학습하고 예측할 경우
 - 예측 결과가 **100%** 정확하다고 나옴
 => `train_test_split()`으로 학습/테스트 데이터 세트를 분리해야 함
- 테스트 데이터 세트를 이용해 예측을 수행한 경우
 - 정확도가 약 **95.56%**

=> 교차 검증

- 과적합 문제: 모델이 학습 데이터에만 과도하게 최적화되어, 실제 예측을 다른 데이터로 수행할 경우에는 예측 성능이 과도하게 떨어지는 것 -> 교차검증 필요
- 교차 검증: 데이터 편종을 막기 위해서 별도의 여러 세트로 구성된 학습 데이터 세트와 검증 데이터 세트에서 학습과 평가를 수행하는 것
- ML 모델의 성능 평가: 교차 검증 기반으로 1차 평가를 한 뒤에 최종적으로 테스트 데이터 세트에 적용해 평가

=> K 폴드 교차 검증

- K개의 데이터 폴드 세트를 만들어서 K번만큼 각 폴드 세트에 학습과 검증 평가를 반복적으로 수행하는 방법
- K개의 예측 평가를 구했으면 이를 평균해서 K 폴드 평가 결과로 반영

=> Stratified K 폴드

- 불균형한 분포도를 가진 레이블(결정 클래스) 데이터 집합을 위한 K 폴드 방식
 - 불균형한 분포도: 정 레이블 값이 특이하게 많거나 매우 적어서 값의 분포가 한쪽으로 치우치는 것
- K 폴드가 레이블 데이터 집합이 원본 데이터 집합의 레이블 분포를 학습/테스트 세트에 제대로 분배하지 못하는 경우의 문제를 해결
- 본 데이터의 레이블 분포를 먼저 고려한 뒤 이 분포와 동일하게 학습과 검증 데이터 세트를 분배함

=> 교차 검증을 보다 간편하게 - `cross_val_score()`

- 내부에서 `Estimator` 학습(`fit`), 예측(`predict`), 평가(`evaluation`)시켜주므로 간단하게 교차 검증을 수행할 수 있음

- `cross_validate()`와 유사함

=) GridSearchCV - 교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에

- 하이퍼 파라미터: 머신러닝 알고리즘을 구성하는 주요 구성 요소, 이를 조정해 알고리즘 성능 개선 가능
 - 6회에 걸쳐 파라미터를 순차적으로 실행
 - 최적의 파라미터와 수행 결과 도출
- GridSearchCV: 교차 검증을 기반으로 하이퍼 파라미터의 최적 값을 찾게 함
 - 사용자가 튜닝하고자 하는 여러 종류의 하이퍼 파라미터를 다양하게 테스트함
 - 시간이 상대적으로 오래 걸림
- 학습 데이터를 GridSearchCV# 이용해 최적 하이퍼 파라미터 튜닝을 수행한 뒤에 별도의 테스트 세트에서 평가하는 것이 일반적인 머신러닝 모델 적용 방법임

—

05. 데이터 전처리

=) 데이터 인코딩

- 레이블 인코딩: 카테고리 피처를 코드형 숫자 값으로 변환하는 것
 - `LabelEncoder` 클래스로 구현
 - `LabelEncoder`을 객체로 생성한 후 `fit()`과 `transform()`을 호출해 레이블 인코딩을 수행
- => 일괄적인 숫자 값으로 변환이 되면서 몇몇 ML 알고리즘에는 이를 적용할 경우 예측 성능이 떨어지는 경우가 발생함

- 원-핫 인코딩: 행 형태로 돼 있는 피처의 고유 값을 열 형태로 차원을 변환한 뒤, 고유 값에 해당하는 칼럼에만 1을 표시하고 나머지 칼럼에는 0을 표시
 - `OneHotEncoder` 클래스로 변환이 가능
 - 입력값으로 2차원 데이터가 필요함
 - `toarray()` 메서드를 이용해 밀집 행렬(Dense Matrix)로 변환해야 함

=) 피처 스케일링과 정규화

- 피처 스케일링: 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업

- 표준화: 데이터의 피쳐 각각이 평균이 0이고 분산이 1인 가우시안 정규 분포를 가진 값으로 변환하는 것
- 정규화: 서로 다른 피쳐의 크기를 통일하기 위해 크기를 변환하는 것

=) StandardScaler

- 표준화를 쉽게 지원하기 위한 클래스
- 개별 피쳐를 평균이 0이고, 분산이 1인 값으로 변환
- SVM, 선형 회귀, 로지스틱 회귀 등에서 중요

=) MinMaxScaler

- 데이터값을 0과 1사이의 범위 값으로 변환
- 데이터의 분포가 가우시안 분포가 아닐 경우에 Min, Max Scale을 적용

=) 학습/테스트 데이터 스케일링 변환 시 유의점

- 테스트 데이터는 학습 데이터의 스케일링 기준에 따라야 함
 - 테스트 데이터의 1값은 학습 데이터와 동일하게 0.1 값으로 변환돼야 함
- 테스트 데이터에 다시 fit()을 적용해서는 안 되며 학습 데이터로 이미 fit()이 적용된 Scaler 객체를 이용해 transform()으로 변환해야 함

CHAPTER 03. 평가

01. 정확도(Accuracy)

=) 정확도

- 실제 데이터에서 예측 데이터가 얼마나 같은지를 판단하는 지표
- 예측 결과가 동일한 데이터 건수 / 전체 예측 데이터 건수
- ML 모델 성능을 왜곡할 수 있음
 - 데이터의 레이블이 불균형한 경우 정확도만으로 모델 성능을 판단 X
 - 여러 가지 분류 지표와 함께 적용해서 성능을 평가해야 함

—

02. 오차행렬(Confusion Matrix, 혼동행렬)

=) 오차행렬

- 예측 오류가 얼마인지와 더불어 어떠한 유형의 예측 오류가 발생하고 있는지 함께 보여주는 지표

=) TP / TN / FP / FN

- **TP (True Positive)**
→ 실제 Positive / Positive라고 예측
- **TN (True Negative)**
→ 실제 Negative / Negative라고 예측
- **FP (False Positive)**
→ 실제 Negative인데 Positive라고 예측
- **FN (False Negative)**
→ 실제 Positive인데 Negative라고 예측

=> 정확도, 정밀도, 재현율의 값을 알 수 있음

03. 정밀도(Precision)와 재현율(Recall)

=) 정밀도(Precision)

- Positive라고 예측한 것 중 실제 Positive인 비율
- 정밀도 = $TP / (FP + TP)$
- = 양성 예측도

=) 정밀도가 중요한 경우

- Positive라고 예측했을 때 틀린 경우를 줄이는 것이 중요한 상황
- ex: 스팸 메일 판단 모델

=) 재현율(Recall)

- 실제 Positive인 것 중 모델이 Positive라고 찾아낸 비율
- 재현율 = $TP / (FN + TP)$
- = 민감도, =TPR

=) 재현율이 중요한 경우

- 실제 Positive를 놓치지 않는 것이 중요한 상황
- ex: 암 판단 모델, 금융 사기 적발 모델

=) 정밀도/재현율 트레이드오프

- 분류의 결정 임계값(Threshold)을 조정해 정밀도 또는 재현율의 수치를 높일 수 있음
-> 어느 한 쪽을 높이면 다른 하나의 수치가 떨어짐
= 정밀도/재현율의 트레이드오프
- 임계값을 낮춘 경우
 - Positive로 예측하는 데이터가 많아짐
 - 재현율이 높아지고, 정밀도는 낮아짐
- 임계값을 높이는 경우
 - positive로 예측하는 데이터가 적어짐
 - 정밀도가 높아지고 재현율이 낮아짐

04. F1 스코어(Score)

=) F1 스코어

- 정밀도와 재현율을 결합한 지표

- $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
- `f1_score()` API 사용
- F1 스코어는 임계값이 0.6일 때 가장 좋은 값을 보여줌
 - 이 경우, 재현율이 크게 감소하므로 주의해야 함

—

05. ROC 곡선과 AUC

=) ROC Curve (수신자 판단 곡선)

- FPR(False Positive Rate) 이 변할 때 TPR(True Positive Rate)이 어떻게 변하는지를 나타내는 곡선
- 가운데 직선에 가까울수록 성능이 떨어지고, 멀어질수록 성능이 뛰어남

=) TPR (=재현율, 민감도)

- $TPR = TP / (TP + FN)$
- $TNR = TN / (FP + TN)$
- $FPR = FP / (FP + TN) = 1 - TNR = 1 - \text{특이성}$

=) AUC

- ROC 곡선 밑의 면적
- 일반적으로 1에 가까울수록 좋은 모델
 - 가운데 직선에서 멀어지고, 왼쪽 상단 모서리 쪽으로 곡선이 이동할수록 1에 가까워짐